

CLASE 1

***INTRODUCCIÓN Y EXPONENCIACIÓN
BINARIA***

Pensar primero, programar después

Taller de Programación Competitiva | 2026

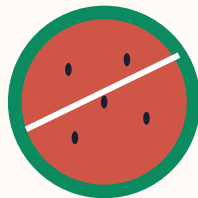


PARTAMOS CON UN PROBLEMA

Watermelon

Tenemos una sandía de peso entero w .
¿Podemos dividirla en **dos partes de peso par positivo**?

- ▶ Entrada: un entero w , con $1 \leq w \leq 100$.
- ▶ Salida: YES si se puede. NO si no.



w kilos

No programemos todavía: busquemos el patrón.

PROBLEMAS CASOS PEQUEÑOS

$$8 = 2 + 6$$

SÍ

$$7$$

NO: es impar

$$2$$

NO: $1 + 1$ no sirve

La observación

La suma de dos pares siempre es par. Y si w es par y mayor que 2, podemos elegir 2 y $w - 2$.

CONDICIÓN

$$w > 2 \quad \text{y} \quad w \bmod 2 = 0$$

DE LA IDEA AL CÓDIGO

Ahora que tenemos la idea, ya estamos listos para programar.

```
1 w = int(input())  
2  
3 if w > 2 and w % 2 == 0:  
4     print("YES")  
5 else:  
6     print("NO")
```

¿PARA QUÉ SIRVE ESTE TALLER?

Eficiencia

Ser consciente de la eficiencia al programar.

Método

Practicar un enfoque metódico de resolución de problemas.

Practicar

Transformar la idea en código robusto bajo presión y aprender del intento.

CONTESTS

ICPC

ENTREVISTAS

PROBLEMAS REALES



PARTE 1 • EFICIENCIA

¿Cómo sabemos si un programa terminará a tiempo?



¿QUÉ SIGNIFICA QUE SEA «RÁPIDO»?

El tiempo real depende del computador, lenguaje, compilador, datos, etc.

En vez de medir segundos exactos, contamos aproximadamente cuántas operaciones realizaremos. Ejemplos de operaciones son: sumas, comparaciones, multiplicaciones, accesos, etc.

La pregunta útil

Si nos ponemos en el peor caso, ¿cuántas operaciones realizaremos?

10^8
operaciones/segundo

Una buena brújula para nuestras estimaciones.

UN EJEMPLO: DOS FOR ANIDADOS

```
1 n = int(input())
2
3 for i in range(n):
4     for j in range(n):
5         print(i, j)
```

- ▶ El primer for da n vueltas.
- ▶ En cada vuelta, el segundo también da n vueltas.
- ▶ La línea interior se ejecuta $n \cdot n = n^2$ veces.

En simple

Decimos que este algoritmo es $\mathcal{O}(n^2)$.

Por simplicidad, conviene quitar las constantes de nuestros cálculos. Por ejemplo, $3n^2$ también se resume como $\mathcal{O}(n^2)$.

LAS RESTRICCIONES SON UNA PISTA

Tamaño aproximado	Complejidad posible
$n \leq 20$	$\mathcal{O}(2^n)$
$n \leq 2\,000$	$\mathcal{O}(n^2)$
$n \leq 2 \cdot 10^5$	$\mathcal{O}(n \log n)$
$n \leq 10^7$	$\mathcal{O}(n)$
$n \leq 10^{18}$	$\mathcal{O}(\log n)$

Idea clave

El límite no es decoración: suele decirnos qué familias de soluciones todavía son posibles.



PARTE 2 • NÚMEROS BINARIOS

El lenguaje de los computadores.



DE BINARIO A DECIMAL

En binario, cada posición representa una potencia de dos. Solo necesitamos los dígitos 0 y 1.

$$(1101)_2 = 1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 13$$

Un bit

Cada dígito binario es un **bit**. Con k bits podemos formar 2^k patrones distintos.

1	1	0	1
2^3	2^2	2^1	2^0
8	4	0	1

DE DECIMAL A BINARIO

Dividimos por 2 y guardamos el resto.

Número	Cociente	Resto
13	6	1
6	3	0
3	1	1
1	0	1

Leemos los restos desde abajo:

$$13 = (1101)_2.$$

¿Por qué es único?

Al dividir por 2, el cociente y el resto son únicos. Como el resto solo puede ser 0 o 1, cada paso determina un único bit.

Una conexión que volverá

$n \% 2$ obtiene el bit.
 $n // 2$ lo descarta.

¿CUÁNTOS BITS CABEN?

En los jueces habituales de programación competitiva:

Tipo	Tamaño habitual	Mayor entero positivo
C++ int	32 bits	$2^{31} - 1 \approx 2 \cdot 10^9$
C++ long long	64 bits	$2^{63} - 1 \approx 9 \cdot 10^{18}$
Python int	tantos bits como necesite	sin un máximo fijo del tipo

¿Entonces Python puede guardar cualquier entero?

El límite termina siendo la memoria disponible. Mientras más bits tenga el número, más memoria ocupa y más lentas se vuelven sus operaciones.



PARTE 3 • EL PROBLEMA

Un exponente gigantesco.



UNA POTENCIA GIGANTE

Problema motivador

Dados a y b , calcula

$$a^b \text{ mód } 998\,244\,353.$$

$$0 \leq a, b \leq 10^{18}$$

Para esta clase definiremos $a^0 = 1$,
incluyendo $0^0 = 1$.

EJEMPLO

$$a = 5, \quad b = 3$$

$$5^3 = 125$$

¿Qué hacemos si $b = 10^{18}$?

CRIPTOGRAFÍA

La exponenciación modular es central en sistemas de clave pública como RSA y Diffie-Hellman.

UN RECORDATORIO: MÓDULO

$x \bmod m$ es el resto al dividir x por m .

$$17 \bmod 5 = 2$$

Nos permite representar respuestas enormes usando valores entre 0 y $m - 1$.

Podemos reducir en cada paso

Si $x' = x \bmod m$ e $y' = y \bmod m$, entonces

$$(x \cdot y) \bmod m = (x' \cdot y') \bmod m.$$

Ejemplo:

$$13 \cdot 12 \bmod 5 = 3 \cdot 2 \bmod 5 = 1.$$

LA SOLUCIÓN DIRECTA

Multiplicamos por a , exactamente b veces.

```
1 def potencia(a, b, mod):  
2     ans = 1  
3     for _ in range(b):  
4         ans = ans * (a % mod) % mod  
5     return ans
```

Complejidad

$$\mathcal{O}(b)$$

El módulo evita que crezca la respuesta, pero no reduce la cantidad de multiplicaciones.

LA RESTRICCIÓN CAMBIA TODO

Solución directa

10^{18} multiplicaciones

$$\frac{10^{18}}{10^8} = 10^{10} \text{ segundos}$$

≈ 317 años

La diferencia no es optimizar detalles: necesitamos **otra idea**.



PARTE 4 • BINEXP

Construyamos la respuesta una pieza a la vez.



EL EXPONENTE NOS DICE QUÉ PIEZAS USAR

Ya sabemos escribir 13 como suma de potencias de dos:

$$13 = (1101)_2 = 8 + 4 + 1.$$

La regla que necesitamos

$$a^{x+y+z} = a^x \cdot a^y \cdot a^z.$$

$$3^{13} = 3^{8+4+1}$$

$$= 3^8 \cdot 3^4 \cdot 3^1$$

La primera idea

La suma en el exponente se transforma en una multiplicación de potencias más fáciles de construir.

LAS POTENCIAS DE DOS SON NUESTRAS PIEZAS

Para calcular a^{13} miramos los bits de $13 = (1101)_2$.

1

a^8

usar

1

a^4

usar

0

a^2

omitir

1

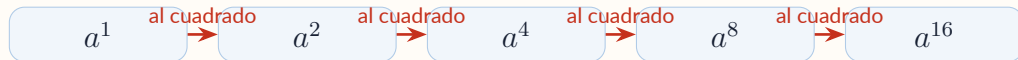
a^1

usar

Las piezas posibles

Basta construir $a^1, a^2, a^4, a^8, \dots$ y multiplicar aquellas cuyo bit sea 1.

UNA MULTIPLICACIÓN CREA LA SIGUIENTE PIEZA



Reutilizar trabajo

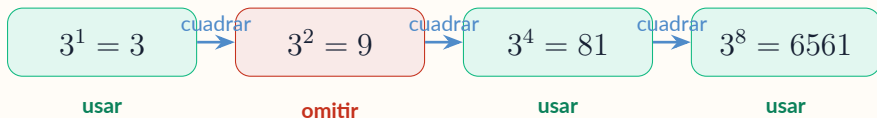
Si ya conocemos a^k , obtenemos a^{2k} con una sola multiplicación:

$$a^{2k} = a^k \cdot a^k.$$

$$\begin{array}{ccccccc} 1 & 2 & 3 & \dots & 60 \\ \text{multiplicación} & & & & \\ a^2 & a^4 & a^8 & \dots & a^{2^{60}} \end{array}$$

Como $2^{60} > 10^{18}$, con cerca de 60 multiplicaciones construimos todas las piezas que podríamos necesitar.

CONSTRUYAMOS 3^{13}



Juntamos las piezas marcadas

$$3^{13} = 3^8 \cdot 3^4 \cdot 3^1 = 6561 \cdot 81 \cdot 3 = 1\,594\,323.$$

EL ALGORITMO EN TRES MOVIMIENTOS

Mantenemos una respuesta, una potencia actual llamada base y los bits que todavía quedan en b .

1. Si el último bit es 1, incorporar base a ans.
2. Cuadrar base para obtener la siguiente pieza.
3. Descartar el bit leído con $b \leftarrow b // 2$.

Repetimos hasta que $b = 0$.

¿Último bit 1? $\Rightarrow$ usar base



$\text{base} \leftarrow \text{base}^2$



$b \leftarrow \lfloor b/2 \rfloor$

SIGAMOS 3^{13}

b en binario	bit	base	acción	ans
$(1101)_2$	1	3	$ans \leftarrow ans \cdot 3$	3
$(110)_2$	0	9	omitir	3
$(11)_2$	1	81	$ans \leftarrow ans \cdot 81$	243
$(1)_2$	1	6561	$ans \leftarrow ans \cdot 6561$	1 594 323
0	—	—	terminar	1 594 323

Resultado

$3^{13} = 1\,594\,323$. Solo hicimos cuatro vueltas del ciclo.

IMPLEMENTACIÓN MODULAR

```
1 def binpow(a, b, mod):
2     base = a % mod
3     ans = 1
4
5     while b > 0:
6         if b % 2 == 1:
7             ans = ans * base % mod
8             base = base * base % mod
9             b //= 2
10
11     return ans
```

BIT 1 $b \% 2$

CUADRAR $\text{base} * \text{base}$

MITAD $b //= 2$

Costo

Tiempo $\mathcal{O}(\log b)$.
Memoria $\mathcal{O}(1)$.

¿POR QUÉ FUNCIONA?

La pieza correcta

Primero, base vale a^1 .

Cada vez que la cuadrados pasa a $a^2, a^4, a^8, \dots$

La decisión correcta

Un bit 1 agrega esa pieza a ans.

Un bit 0 la omite.

Todos los bits

En cada vuelta descartamos un bit. Por eso cada posición se procesa exactamente una vez.

Al terminar, ans contiene el producto de todas las piezas marcadas:

$$a^{b_{\text{original}} \bmod m}.$$

CASOS BORDE Y TRAMPAS

Antes de enviar

- ▶ $b = 0$ debe devolver 1.
- ▶ Inicializar base como $a \bmod m$.
- ▶ Usar división entera: $b \neq 2$.
- ▶ No olvidar el módulo tras cada producto.

El módulo va dentro del ciclo

Aplicarlo después de cada producto mantiene pequeños los valores sin cambiar la respuesta final.

Así evitamos overflow en C++ y enteros innecesariamente grandes en Python.

LA IDEA ES MÁS GRANDE QUE LAS POTENCIAS

Binexp solo necesita:

- ▶ una operación asociativa.
- ▶ un elemento identidad.
- ▶ poder combinar un objeto consigo mismo.

Por eso la misma estructura puede elevar:

números, matrices, permutaciones.

Desafío avanzado

Si una matriz transforma un vector una vez, ¿qué representa elevarla a la potencia n ?

Pista

Aplicar la transformación n veces. Esa será la idea detrás del último problema del contest.



A PRACTICAR

